

INTRODUCCIÓN A LA MAQUETACIÓN I

HTML: estructura, etiquetas y buenas prácticas

Apuntes revisados y ampliados

Documento mejorado a partir de las notas originales

1. HTML

HTML son las siglas de **Hypertext Markup Language** (lenguaje de marcado de hipertexto). La declaración `<!DOCTYPE html>` indica al navegador que el documento es de tipo HTML.

En cuanto a la apertura y el cierre de las etiquetas, se pueden diferenciar dos tipos:

- **Bloques estructurales:** se componen de una etiqueta de apertura y otra de cierre idénticas; la etiqueta de cierre incorpora una barra /. Este tipo de etiquetas puede incluir contenido en su interior.
- **Sin cierre dependiente (etiquetas vacías):** no requieren contenido interior; se definen mediante atributos, por ejemplo `<meta/>`.

★ Lo más importante

- ▶ HTML = **H**ypertext **M**arkup **L**anguage.
- ▶ Toda página HTML debe empezar con `<!DOCTYPE html>`.
- ▶ Existen dos tipos de etiquetas: las que abren y cierran (con contenido) y las que no requieren cierre (etiquetas vacías, definidas por atributos).

2. Etiquetas <Tags>

2.1. Etiqueta base

La etiqueta `<base>` permite definir la **ruta base** para todas las rutas absolutas del documento. No debe usarse para definir rutas relativas, ya que estas se calculan respecto a la posición del propio archivo.

```

```



Ejemplo de uso de la etiqueta <base>

2.2. Etiqueta meta (viewport)

La etiqueta `<meta name="viewport">` define el **viewport** (el área visible de un sitio web para el usuario). **Es fundamental y siempre tiene que estar presente.**

Además, permite definir cómo debe adaptarse o comportarse el contenido en función del dispositivo, pudiendo establecer su ancho, alto y escala inicial.

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```



Etiqueta meta viewport

2.3. Etiqueta link

Se usa principalmente para vincular **hojas de estilo CSS**, aunque tiene otros usos (por ejemplo, iconos o fuentes).

```
<link rel="stylesheet" href="guide.css">
```



Etiqueta link

2.4. Etiqueta main

La etiqueta `<main>` sirve para albergar el contenido principal de la página web.

Pueden existir múltiples footer; de hecho, es muy habitual incorporar uno dentro de otras etiquetas estructurales como section, aside o article. Lo más común es tener un footer que se repite en todas las páginas y que, por regla general, incorpora la información de contacto o ciertos enlaces del sitio.

★ Regla clave

- Si un elemento se repite en varias páginas, **nunca será `<main>`**.

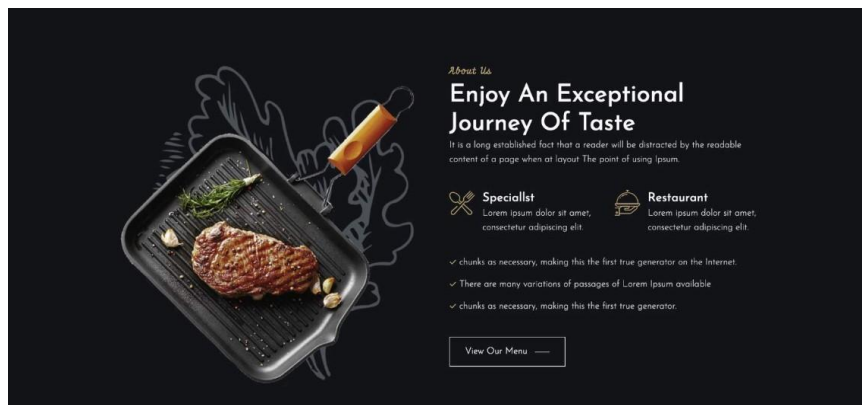
3. Article vs Section

La etiqueta `<article>` se utiliza para un **elemento autocontenido** que tenga intención de ser distribuible o reutilizable.

La etiqueta `<section>` representa una **sección genérica** de un documento.

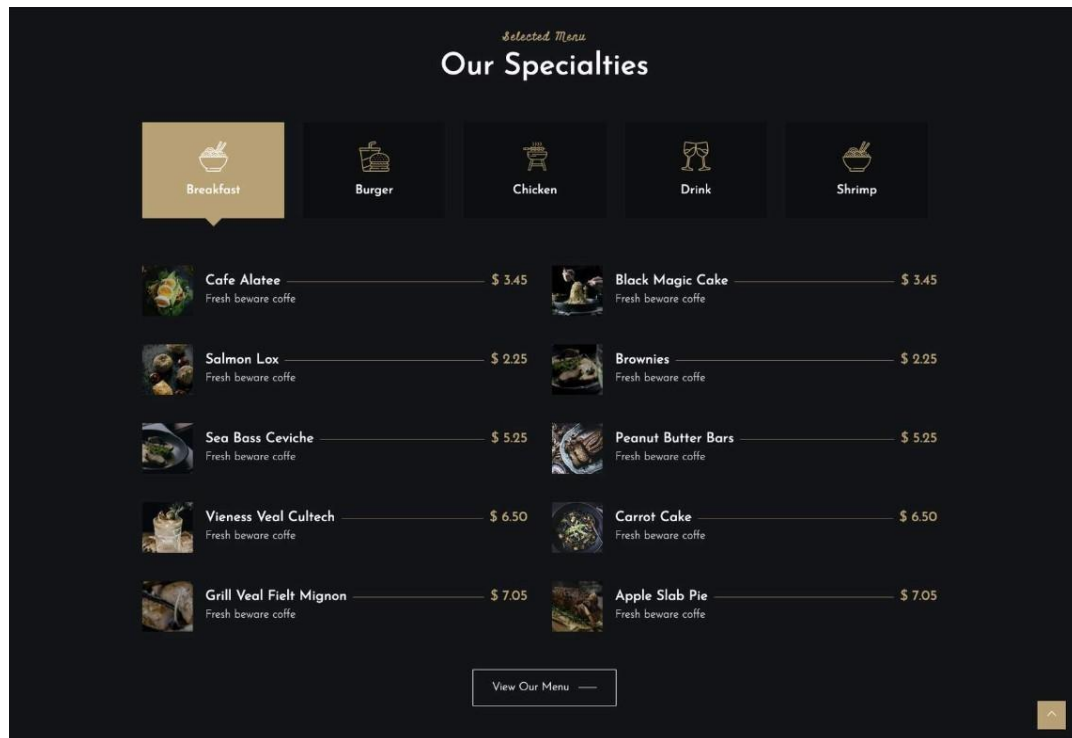
3.1. Section

El contenido de la captura se podría encapsular perfectamente en un section que tuviese un header (con el título y el subtítulo), un footer (con el botón que enlaza con el menú) y el resto del contenido, que podría ir encapsulado en un div.



Ejemplo de estructura con `<section>`

3.2. Article



Ejemplo de estructura con <article>

En este caso sí tiene sentido que sea un article, ya que muy probablemente se repita en otras páginas del sitio web, y podría ser distribuido y compartido por sí mismo (por ejemplo, para que un amigo pudiera consultar la carta).

El elemento <nav> se utiliza para el menú general de un sitio web; también puede emplearse en otros bloques de navegación, en este caso para navegar entre las distintas categorías de la carta.

★ Cómo decidir entre article y section

- ▶ ¿El contenido tiene sentido por sí solo y podría compartirse de forma independiente? → article.
- ▶ ¿Es simplemente una sección temática dentro de un documento más amplio? → section.

4. Atributos

Un atributo es un modificador que permite añadir características a las etiquetas HTML. Los más habituales son:

- **href / src:** indican la URL de un recurso, ya sea asociado (href) o integrado (src).
- **width:** indica el ancho de algunos elementos.
- **alt:** permite incluir un texto alternativo que describe el elemento.
- **class / id:** identificadores de elementos.

En la mayoría de los atributos existen dos partes: la de contenido y la de IDL.

- **Atributo de contenido:** se define desde el HTML, se consulta mediante `getAttribute()` y siempre es de tipo string.
- **Atributo IDL:** se utiliza como propiedad de JavaScript para consultar o modificar características del elemento.

Los atributos `class` e `id` se usan como selectores, tanto para definir estilos CSS como para seleccionar elementos desde JavaScript:

- **class:** puede repetirse a lo largo de la página. Permite asociar una serie de características CSS y replicarlas en múltiples elementos (por ejemplo, un modificador de encabezados), manteniendo la semántica correcta y facilitando cambios de estilo.
- **id:** tiene que ser **único**; no se puede repetir el mismo id en distintos elementos de la misma página.

Siguiendo una nomenclatura tipo BEM:

- **Bloque:** es la propia clase del elemento.
- **Elemento:** compone un bloque y no puede utilizarse de manera separada; describe el propósito del bloque.
- **Modificador:** se utiliza para describir apariencia, estado o comportamiento.

★ Lo más importante

- ▶ **class** se puede repetir; **id** debe ser único en toda la página.
- ▶ El atributo de contenido (HTML/`getAttribute`) y el atributo IDL (JavaScript) no son lo mismo.

5. Formularios

5.1. Form

La etiqueta `<form>` es **fundamental incluirla como contenedor de cualquier formulario** para que pueda ser identificado como tal.

Entre sus atributos más importantes:

- **action:** la URI a la que se quiere enviar el formulario.
- **enctype:** necesario cuando se quiere mandar un archivo; debe marcarse como `'multipart/form-data'`.
- **method:** puede asignarse:
 - `post` — los datos se envían en el cuerpo del formulario; apto para el envío de ficheros.

5.2. Input

La etiqueta `<input>` sirve para interactuar con el usuario y permitirle la entrada de datos.

Cualquier información de entrada del usuario puede ser sensible y **hay que tratarla con el máximo cuidado** posible para que no pueda ser extraída por terceros. También hay que asumir que el usuario puede

intentar hacer un mal uso de estas fuentes de entrada, por lo que conviene poner todas las barreras posibles.

Algunas acciones recomendadas:

- **Configurar correctamente los inputs:** añadir atributos de validación (required, pattern) y definir el tipo adecuado según el dato (email, password, etc.), reforzando las reglas con JavaScript si es necesario. Mejora la seguridad y la usabilidad.
- **Añadir un captcha** en el formulario para evitar envíos masivos o automatizados.
- **Incluir un campo oculto** con un token único generado en el servidor, para evitar ataques **CSRF** (Cross-Site Request Forgery).

★ Lo más importante de Formularios

- ▶ `<form>` es obligatorio como contenedor de cualquier formulario.
- ▶ `method="post"` para archivos y datos sensibles; `method="get"` solo para datos ASCII no sensibles.
- ▶ Todo dato de entrada del usuario es potencialmente sensible: valida, usa captcha y protege contra CSRF con un token.